

온라인 쇼핑몰 운영 아키텍처 문서 v1

0. 문서 목적

이 문서는 Spring Boot 기반 온라인 쇼핑몰을 단순 CRUD 서비스가 아니라 **운영 가능한 주문·결제·재고·배송 시스템**으로 설계하기 위한 아키텍처 문서이다.

쇼핑몰에서 가장 중요한 것은 단순히 상품을 보여주고 주문을 저장하는 것이 아니다. 실제 운영 환경에서는 다음 문제가 반드시 발생한다.

- 여러 사용자가 동시에 같은 상품을 주문한다.
- 결제 요청은 성공했지만 주문 상태 변경이 실패한다.
- PG사 콜백이 중복으로 들어온다.
- 주문 취소와 배송 시작이 동시에 발생한다.
- 대량 주문, 대량 배송 상태 변경, 대량 통계 집계 필요하다.
- 상품 목록, 주문 목록, 리뷰 목록 조회가 느려진다.
- 캐시와 DB 데이터가 불일치할 수 있다.
- 장애 발생 후 어떤 주문을 재처리해야 하는지 추적해야 한다.

따라서 이 문서의 목적은 다음과 같다.

1. 온라인 쇼핑몰의 핵심 도메인을 정의한다.
2. 주문, 결제, 재고, 배송의 상태 변경 흐름을 명확히 한다.
3. 동시성 환경에서 데이터 정합성을 지키는 구조를 설계한다.
4. 조회 성능과 쓰기 성능을 함께 고려한다.
5. 장애 복구, 재처리, 모니터링까지 포함한 운영 가능한 구조를 만든다.

1. 전체 시스템 개요

1-1. 핵심 도메인

온라인 쇼핑몰의 핵심 도메인은 다음과 같다.

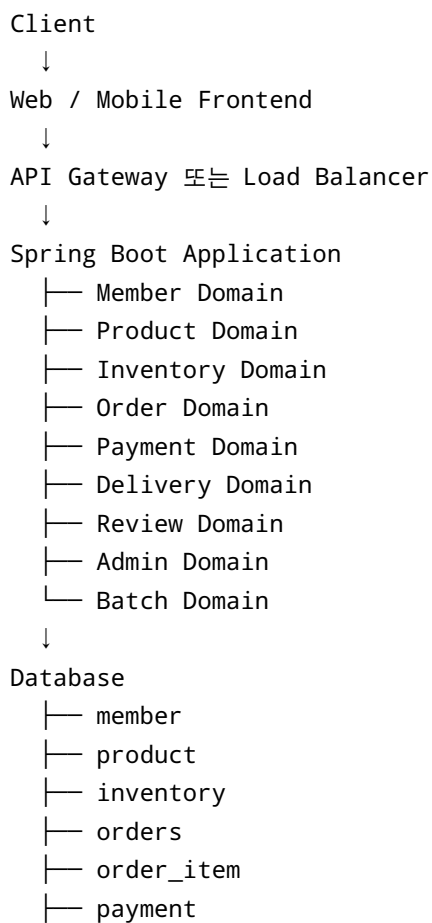
도메인	설명
Member	회원, 인증, 권한
Product	상품, 가격, 판매 상태
Inventory	재고, 예약 재고, 판매 가능 재고
Cart	장바구니
Order	주문, 주문 상태
OrderItem	주문 당시 상품 스냅샷
Payment	결제 요청, 승인, 실패, 취소

도메인	설명
Delivery	배송지, 배송 상태
Review	리뷰, 평점
Coupon	쿠폰 발급, 사용
Point	포인트 적립, 사용
Settlement	정산, 매출 집계
Batch	대량 처리, 통계, 만료 처리
Admin	운영자 관리 기능
Monitoring	로그, 지표, 장애 추적

처음 개발할 때는 Member, Product, Order, OrderItem, Payment, Delivery, Review 중심으로 시작하고, 이후 Coupon, Point, Settlement, Batch를 확장한다.

2. 운영 관점의 전체 아키텍처

2-1. 논리 아키텍처



- └─ delivery
- └─ review
- └─ coupon
- └─ point
- └─ outbox_event
- └─ order_event_log
- └─ product_stat

External Systems

- └─ PG사 결제 API
- └─ 배송사 API
- └─ 이메일/SMS 알림
- └─ 관리자 시스템

Infrastructure

- └─ Redis
- └─ Message Queue
- └─ Object Storage
- └─ Monitoring
- └─ Logging

2-2. 주요 인프라 구성

구성 요소	역할
Spring Boot	비즈니스 API 서버
RDBMS	주문, 결제, 재고 등 정합성이 중요한 데이터 저장
Redis	캐시, 분산 락, 임시 선점 데이터
Message Queue	결제 후처리, 알림, 통계, 비동기 이벤트 처리
Object Storage	상품 이미지, 리뷰 이미지 저장
Batch Server	통계 집계, 만료 처리, 대량 상태 변경
Monitoring	API 응답 시간, 에러율, DB 상태, 락 대기 시간 추적
Logging	주문, 결제, 장애 추적용 로그 저장

3. 도메인 설계 원칙

3-1. 주문과 상품은 ManyToMany로 설계하지 않는다

주문과 상품은 겹으로 보면 다대다 관계이다.

```
Order N : M Product
```

하지만 실무에서는 반드시 중간 엔티티인 `OrderItem` 을 둔다.

```
Order 1 : N OrderItem
Product 1 : N OrderItem
```

이유는 주문상품에 단순 연결 정보만 있는 것이 아니라, 주문 당시의 중요한 정보가 들어가기 때문이다.

```
order_item
- order_item_id
- order_id
- product_id
- product_name_snapshot
- order_price
- quantity
- discount_amount
- option_name_snapshot
```

상품명이 나중에 바뀌어도 과거 주문 내역의 상품명은 바뀌면 안 된다.
상품 가격이 나중에 바뀌어도 과거 주문 금액은 바뀌면 안 된다.

따라서 주문 내역에는 반드시 **주문 당시 상품 정보 스냅샷**을 저장한다.

3-2. 상품과 재고를 분리한다

초기에는 `product.stock_quantity` 하나로 시작할 수 있다.

```
product
- product_id
- name
- price
- stock_quantity
```

하지만 운영 수준에서는 상품 정보와 재고 정보를 분리하는 것이 좋다.

```
product
- product_id
- name
- price
- status
- category_id
```

```
- created_at
- updated_at
- deleted_at
```

```
inventory
- product_id
- total_quantity
- reserved_quantity
- available_quantity
- version
- updated_at
```

판매 가능 재고는 다음처럼 계산할 수 있다.

```
available_quantity = total_quantity - reserved_quantity
```

단순 쇼핑몰에서는 `stock_quantity` 만으로 충분할 수 있다.

하지만 결제 전 재고 예약, 결제 실패 시 재고 복구, 선착순 이벤트를 고려하면 `Inventory` 를 별도 도메인으로 분리하는 것이 더 안전하다.

4. 주문 상태 머신

4-1. 주문 상태

주문은 단순히 생성되고 취소되는 데이터가 아니다.

주문은 상태를 가진다.

PENDING_PAYMENT	결제 대기
PAID	결제 완료
PAYMENT_FAILED	결제 실패
PREPARING	상품 준비 중
SHIPPED	배송 중
DELIVERED	배송 완료
CONFIRMED	구매 확정
CANCEL_REQUESTED	취소 요청
CANCELLED	주문 취소
REFUND_REQUESTED	환불 요청
REFUNDED	환불 완료

4-2. 주문 상태 전이

PENDING_PAYMENT

- PAID
- PAYMENT_FAILED
- CANCELLED

PAID

- PREPARING
- CANCEL_REQUESTED
- REFUND_REQUESTED

PREPARING

- SHIPPED
- CANCEL_REQUESTED

SHIPPED

- DELIVERED

DELIVERED

- CONFIRMED
- REFUND_REQUESTED

CANCEL_REQUESTED

- CANCELLED

REFUND_REQUESTED

- REFUNDED

4-3. 상태별 가능한 행위

현재 상태	가능한 행위
PENDING_PAYMENT	결제 승인, 결제 실패, 주문 만료
PAID	배송 준비, 주문 취소 요청
PREPARING	배송 시작, 주문 취소 요청
SHIPPED	배송 완료 처리
DELIVERED	구매 확정, 반품/환불 요청
CONFIRMED	리뷰 작성
CANCELLED	추가 변경 불가
REFUNDED	추가 변경 불가

4-4. 상태 전이 검증 예시

```
public void pay() {  
    if (this.status != OrderStatus.PENDING_PAYMENT) {  
        throw new IllegalStateException("결제 대기 상태에서만 결제 완료 처리가 가능합니다.");  
    }  
    this.status = OrderStatus.PAID;  
}  
  
public void cancel() {  
    if (this.status == OrderStatus.SHIPPED || this.status == OrderStatus.DELIVERED) {  
        throw new IllegalStateException("배송 시작 이후에는 일반 취소가 불가능합니다.");  
    }  
    if (this.status == OrderStatus.CANCELLED) {  
        throw new IllegalStateException("이미 취소된 주문입니다.");  
    }  
    this.status = OrderStatus.CANCELLED;  
}
```

상태 전이 규칙은 Controller나 Service에 흩어놓지 말고, 가능하면 `Order` 도메인 내부에 둔다.

5. 주문 생성 아키텍처

5-1. 기본 주문 생성 흐름

```
사용자 주문 요청  
↓  
회원 검증  
↓  
상품 검증  
↓  
판매 가능 상태 검증  
↓  
재고 예약 또는 재고 차감  
↓  
주문 PENDING_PAYMENT 생성  
↓  
주문상품 생성  
↓  
배송정보 생성  
↓  
결제 요청 준비
```

↓
응답 반환

5-2. 주문 생성 시 트랜잭션 범위

하나의 DB 트랜잭션 안에서 처리할 것:

- 주문 생성
- 주문상품 생성
- 배송정보 생성
- 재고 예약 또는 재고 차감

트랜잭션 밖에서 처리할 것:

- 외부 PG 결제 API 호출
- 이메일 발송
- 문자 발송
- 외부 배송사 API 호출

외부 API 호출을 DB 트랜잭션 안에서 처리하면 위험하다.

이유는 다음과 같다.

- 외부 API 응답 지연 동안 DB 커넥션을 점유한다.
- 재고 row lock이 오래 유지될 수 있다.
- 다른 주문 요청이 대기한다.
- PG 장애가 DB 장애로 전파될 수 있다.

따라서 주문 생성은 다음처럼 나눈다.

1. DB 트랜잭션 시작
2. 주문 PENDING_PAYMENT 생성
3. 재고 예약
4. DB 트랜잭션 커밋
5. PG 결제 요청
6. 결제 승인 결과 또는 콜백 수신
7. 주문 상태 PAID 변경

6. 재고 아키텍처

6-1. 재고 차감 방식 선택 기준

상황	권장 방식
일반 상품 주문	조건부 UPDATE
충돌이 적은 상품 수정	낙관적 락
정합성이 매우 중요한 단건 처리	비관적 락
인기 상품 선착순 판매	Redis 선점 + DB 확정
결제 전 재고 확보 필요	재고 예약 테이블 또는 reserved_quantity
여러 상품을 한 번에 주문	product_id 오름차순으로 락 획득

6-2. 조건부 UPDATE 방식

재고 차감에서 가장 실용적인 방식은 조건부 UPDATE다.

```
UPDATE inventory
SET available_quantity = available_quantity - :quantity
WHERE product_id = :productId
AND available_quantity >= :quantity;
```

성공 여부는 변경된 row 수로 판단한다.

```
updated row = 1 → 재고 차감 성공
updated row = 0 → 재고 부족
```

JPA 예시:

```
@Modifying
@Query("""
    update Inventory i
    set i.availableQuantity = i.availableQuantity - :quantity
    where i.productId = :productId
    and i.availableQuantity >= :quantity
""")
int decreaseStock(
    @Param("productId") Long productId,
```

```
@Param("quantity") int quantity  
);
```

사용 예시:

```
@Transactional  
public void decreaseStock(Long productId, int quantity) {  
    int updated = inventoryRepository.decreaseStock(productId, quantity);  
  
    if (updated == 0) {  
        throw new IllegalStateException("재고가 부족합니다.");  
    }  
}
```

6-3. 재고 예약 방식

결제 전 재고를 확보해야 한다면 단순 차감보다 예약 방식이 좋다.

```
사용자 주문 요청  
↓  
재고 예약  
↓  
주문 PENDING_PAYMENT 생성  
↓  
결제 요청  
↓  
결제 성공 → 예약 재고 확정 차감  
↓  
결제 실패 → 예약 재고 해제
```

재고 테이블 예시:

```
inventory  
- product_id  
- total_quantity  
- reserved_quantity  
- available_quantity
```

재고 예약 SQL:

```
UPDATE inventory  
SET reserved_quantity = reserved_quantity + :quantity,
```

```
available_quantity = available_quantity - :quantity
WHERE product_id = :productId
AND available_quantity >= :quantity;
```

결제 실패 시 예약 해제:

```
UPDATE inventory
SET reserved_quantity = reserved_quantity - :quantity,
    available_quantity = available_quantity + :quantity
WHERE product_id = :productId;
```

결제 성공 시 예약 확정:

```
UPDATE inventory
SET reserved_quantity = reserved_quantity - :quantity,
    total_quantity = total_quantity - :quantity
WHERE product_id = :productId;
```

6-4. 재고 예약 만료 배치

결제 대기 상태가 오래 지속되면 예약 재고를 해제해야 한다.

대상:

- 주문 상태 = PENDING_PAYMENT
- 생성 후 10분 이상 경과
- 결제 승인 내역 없음

처리:

- 주문 상태 PAYMENT_FAILED 또는 CANCELLED 변경
- 예약 재고 해제
- 처리 이력 저장

배치 예시:

```
PendingPaymentExpirationJob
↓
PENDING_PAYMENT 주문 조회
↓
결제 승인 여부 확인
↓
예약 재고 해제
↓
```

주문 상태 만료 처리



처리 로그 저장

6-5. 데드락 방지

여러 상품을 동시에 주문할 때는 항상 같은 순서로 재고를 차감한다.

나쁜 예시:

트랜잭션 1: 상품 A 락 → 상품 B 락

트랜잭션 2: 상품 B 락 → 상품 A 락

좋은 예시:

항상 product_id 오름차순으로 재고 차감

```
List<OrderItemRequest> sortedItems = request.getItems().stream()
    .sorted(Comparator.comparing(OrderItemRequest::getProductId))
    .toList();
```

7. 결제 아키텍처

7-1. 결제는 외부 시스템 연동이다

결제는 단순히 `payment INSERT`가 아니다.

PG사라는 외부 시스템과 상태를 맞춰야 하는 작업이다.

실무에서 결제는 다음 문제가 발생한다.

- 결제 요청이 중복으로 들어온다.
- PG 콜백이 중복으로 들어온다.
- 결제는 성공했지만 서버 응답이 실패한다.
- 서버는 실패로 알았지만 PG에는 성공으로 기록된다.
- 주문 취소와 결제 승인 콜백이 동시에 들어온다.
- 부분 취소가 필요하다.
- 환불이 실패할 수 있다.

따라서 결제 도메인은 반드시 **역등성**과 **상태 동기화**를 고려해야 한다.

7-2. payment 테이블

```
payment
- payment_id
- order_id
- payment_key
- pg_provider
- pg_transaction_id
- idempotency_key
- payment_status
- requested_amount
- approved_amount
- cancelled_amount
- approved_at
- failed_at
- cancelled_at
- failure_code
- failure_message
- created_at
- updated_at
```

7-3. 결제 상태

READY	결제 준비
REQUESTED	결제 요청
APPROVED	결제 승인
FAILED	결제 실패
CANCELLED	전체 취소
PARTIAL_CANCELLED	부분 취소
REFUNDED	환불 완료

7-4. 결제 멍등성

같은 주문에 대해 결제 승인 요청이 중복으로 들어와도 결제는 한 번만 처리되어야 한다.

이를 위해 `idempotency_key` 를 저장한다.

```
idempotency_key = order_no + payment_attempt_no
```

DB 제약조건:

```
ALTER TABLE payment
ADD CONSTRAINT uk_payment_idempotency_key UNIQUE(idempotency_key);
```

처리 원칙:

이미 같은 idempotency_key로 APPROVED 된 결제가 있으면
새로 승인하지 않고 기존 결과를 반환한다.

7-5. PG 콜백 중복 처리

PG 콜백은 중복으로 들어올 수 있다.
따라서 콜백 처리도 멍등해야 한다.

```
@Transactional
public void handlePaymentCallback(PaymentCallbackRequest request) {
    Payment payment = paymentRepository.findByPaymentKey(request.getPaymentKey())
        .orElseThrow(() -> new IllegalArgumentException("결제 정보가 없습니다."));

    if (payment.isApproved()) {
        return;
    }

    payment.approve(request.getApprovedAmount(), request.getPgTransactionId());

    Order order = orderRepository.findById(payment.getOrderId())
        .orElseThrow();

    order.pay();
}
```

핵심은 다음과 같다.

이미 처리된 콜백은 다시 처리하지 않는다.
중복 콜백은 성공 응답으로 무시한다.

7-6. 결제 성공 후 주문 상태 변경 실패 대응

문제 상황:

1. PG 결제 승인 성공
2. 서버에서 payment APPROVED 저장 성공
3. order 상태 PAID 변경 실패

이 경우 결제와 주문 상태가 불일치한다.

이를 해결하기 위해 **결제 상태 동기화 배치**가 필요하다.

```
PaymentReconciliationJob
↓
payment = APPROVED
↓
order != PAID
↓
PG 승인 내역 재확인
↓
order 상태 PAID로 보정
↓
보정 이력 저장
```

7-7. 결제 실패 처리

결제 실패 시 처리할 것:

- payment 상태 FAILED 변경
- failure_code 저장
- failure_message 저장
- 주문 상태 PAYMENT_FAILED 변경
- 예약 재고 해제

실패 코드 저장은 중요하다.

운영자가 장애 원인을 분석할 수 있어야 하기 때문이다.

7-8. 결제 취소와 환불

결제 취소는 주문 취소와 연결된다.

```
사용자 주문 취소 요청
↓
주문 상태 확인
↓
```

배송 시작 전인지 확인
↓
PG 결제 취소 요청
↓
결제 취소 성공
↓
주문 CANCELLED 변경
↓
재고 복구

주의할 점:

PG 결제 취소도 실패할 수 있다.
결제 취소 요청도 중복될 수 있다.
부분 취소가 필요할 수 있다.

따라서 취소 요청에도 `cancel_idempotency_key` 를 둔다.

8. 배송 아키텍처

8-1. 배송 상태

READY	배송 준비
PREPARING	상품 준비 중
SHIPPED	배송 중
DELIVERED	배송 완료
RETURNING	반품 중
RETURNED	반품 완료

8-2. 배송 상태 전이

READY
→ PREPARING
→ SHIPPED
→ DELIVERED

DELIVERED
→ RETURNING
→ RETURNED

8-3. 주문 취소와 배송 상태

주문 취소 가능 여부는 배송 상태와 연결된다.

배송 상태	주문 취소
READY	가능
PREPARING	정책에 따라 가능
SHIPPED	일반 취소 불가
DELIVERED	반품/환불 프로세스로 전환

따라서 주문 취소 시 단순히 주문 상태만 보면 안 된다.
배송 상태를 함께 확인해야 한다.

9. 리뷰 아키텍처

9-1. 리뷰 작성 조건

리뷰는 아무 상품에나 작성할 수 없다.

리뷰 작성 조건:

- 로그인한 회원이어야 한다.
- 실제 구매한 상품이어야 한다.
- 주문 상태가 DELIVERED 또는 CONFIRMED 여야 한다.
- 하나의 order_item에는 하나의 리뷰만 작성할 수 있다.
- 삭제된 상품에는 리뷰를 작성할 수 없다.

9-2. 리뷰 중복 방지

애플리케이션에서 먼저 조회해서 막는 것만으로는 부족하다.

동시에 두 요청이 들어오면 다음 문제가 발생할 수 있다.

- A 요청: 리뷰 없음 확인
- B 요청: 리뷰 없음 확인
- A 요청: 리뷰 INSERT
- B 요청: 리뷰 INSERT

따라서 DB 유니크 제약조건이 필요하다.

```
ALTER TABLE review
ADD CONSTRAINT uk_review_order_item UNIQUE(order_item_id);
```

9-3. 리뷰 통계 갱신

상품 상세 화면에서는 보통 다음 정보를 보여준다.

- 평균 평점
- 리뷰 수
- 평점 분포

매번 `review` 테이블을 집계하면 느려질 수 있다.

따라서 `product_stat` 테이블을 둔다.

```
product_stat
- product_id
- review_count
- average_rating
- rating_1_count
- rating_2_count
- rating_3_count
- rating_4_count
- rating_5_count
- sales_count
- updated_at
```

리뷰 작성 시 실시간으로 갱신하거나, 이벤트 기반으로 비동기 갱신할 수 있다.

10. 데이터베이스 설계

10-1. 주요 테이블

```
member
- member_id
- email
- password
- name
- role
- status
- created_at
```

- updated_at
- deleted_at

product

- product_id
- name
- price
- status
- category_id
- thumbnail_url
- created_at
- updated_at
- deleted_at

inventory

- product_id
- total_quantity
- reserved_quantity
- available_quantity
- version
- updated_at

orders

- order_id
- order_no
- member_id
- order_status
- total_price
- ordered_at
- paid_at
- cancelled_at
- created_at
- updated_at

order_item

- order_item_id
- order_id
- product_id
- product_name_snapshot
- order_price
- quantity
- discount_amount
- option_name_snapshot

payment

- payment_id
- order_id
- payment_key

- pg_provider
- pg_transaction_id
- idempotency_key
- payment_status
- requested_amount
- approved_amount
- cancelled_amount
- approved_at
- failed_at
- cancelled_at
- failure_code
- failure_message
- created_at
- updated_at

delivery

- delivery_id
- order_id
- receiver_name
- receiver_phone
- address
- delivery_status
- tracking_number
- shipped_at
- delivered_at

review

- review_id
- member_id
- product_id
- order_item_id
- rating
- content
- created_at
- updated_at
- deleted_at

10-2. 필수 제약조건

```
ALTER TABLE member
ADD CONSTRAINT uk_member_email UNIQUE(email);
```

```
ALTER TABLE orders
ADD CONSTRAINT uk_orders_order_no UNIQUE(order_no);
```

```

ALTER TABLE payment
ADD CONSTRAINT uk_payment_key UNIQUE(payment_key);

ALTER TABLE payment
ADD CONSTRAINT uk_payment_idempotency_key UNIQUE(idempotency_key);

ALTER TABLE review
ADD CONSTRAINT uk_review_order_item UNIQUE(order_item_id);

```

수량 관련 제약조건:

```

ALTER TABLE order_item
ADD CONSTRAINT chk_order_item_quantity CHECK (quantity > 0);

ALTER TABLE inventory
ADD CONSTRAINT chk_inventory_quantity CHECK (
    total_quantity >= 0
    AND reserved_quantity >= 0
    AND available_quantity >= 0
);

```

11. API 설계

11-1. 회원 API

Method	URI	설명
POST	/api/members	회원가입
POST	/api/login	로그인
GET	/api/members/me	내 정보 조회
PATCH	/api/members/me	내 정보 수정
DELETE	/api/members/me	회원 탈퇴

11-2. 상품 API

Method	URI	설명
GET	/api/products	상품 목록 조회
GET	/api/products/{productId}	상품 상세 조회
GET	/api/products/{productId}/reviews	상품 리뷰 목록

Method	URI	설명
POST	/api/admin/products	상품 등록
PATCH	/api/admin/products/{productId}	상품 수정
DELETE	/api/admin/products/{productId}	상품 삭제

11-3. 주문 API

Method	URI	설명
POST	/api/orders	주문 생성
GET	/api/orders	내 주문 목록 조회
GET	/api/orders/{orderId}	주문 상세 조회
POST	/api/orders/{orderId}/cancel	주문 취소
POST	/api/orders/{orderId}/confirm	구매 확정

11-4. 결제 API

Method	URI	설명
POST	/api/payments/ready	결제 준비
POST	/api/payments/approve	결제 승인
POST	/api/payments/callback	PG 콜백 수신
POST	/api/payments/{paymentId}/cancel	결제 취소

11-5. 관리자 API

Method	URI	설명
GET	/api/admin/orders	주문 검색
GET	/api/admin/payments	결제 검색
GET	/api/admin/products	상품 관리 목록
POST	/api/admin/products/bulk-upload	상품 대량 업로드
GET	/api/admin/batch-jobs	배치 실행 이력
POST	/api/admin/batch-jobs/{jobName}/retry	배치 재실행

12. 조회 성능 최적화

12-1. 상품 목록 조회

상품 목록은 읽기 요청이 많다.

```
GET /api/products?categoryId=1&sort=latest&cursor=100&size=20
```

조회 쿼리:

```
SELECT product_id, name, price, thumbnail_url
FROM product
WHERE category_id = ?
  AND status = 'SELLING'
  AND deleted_at IS NULL
  AND (created_at, product_id) < (?, ?)
ORDER BY created_at DESC, product_id DESC
LIMIT 20;
```

인덱스:

```
CREATE INDEX ix_product_category_status_created
ON product(category_id, status, created_at DESC, product_id DESC);
```

12-2. 주문 목록 조회

주문 목록은 Offset Pagination보다 Keyset Pagination이 적합하다.

```
SELECT order_id, order_no, order_status, total_price, ordered_at
FROM orders
WHERE member_id = ?
  AND (ordered_at, order_id) < (?, ?)
ORDER BY ordered_at DESC, order_id DESC
LIMIT 20;
```

인덱스:

```
CREATE INDEX ix_orders_member_ordered
ON orders(member_id, ordered_at DESC, order_id DESC);
```

12-3. 리뷰 목록 조회

```
SELECT review_id, member_id, rating, content, created_at
FROM review
WHERE product_id = ?
  AND deleted_at IS NULL
  AND (created_at, review_id) < (?, ?)
ORDER BY created_at DESC, review_id DESC
LIMIT 20;
```

인덱스:

```
CREATE INDEX ix_review_product_created
ON review(product_id, created_at DESC, review_id DESC);
```

12-4. N+1 문제 해결

주문 목록 조회 시 다음 문제가 발생할 수 있다.

```
주문 목록 조회 1번
주문1의 주문상품 조회 1번
주문2의 주문상품 조회 1번
주문3의 주문상품 조회 1번
...
```

해결 기준:

```
ToOne 관계
→ fetch join 우선 고려

ToMany 관계
→ batch size 또는 별도 조회 후 애플리케이션에서 묶기

복잡한 화면 조회
→ DTO 직접 조회 또는 QueryDSL 사용
```


13. 쓰기 성능 최적화

13-1. 대량 INSERT

상품을 100만 건 등록할 때 JPA `save()` 를 반복하면 위험하다.

개선 흐름:

```
CSV 업로드
↓
staging_product 테이블 적재
↓
데이터 검증
↓
정상 데이터만 product 반영
↓
실패 데이터 error 테이블 저장
```

13-2. 대량 UPDATE

나쁜 예시:

```
UPDATE orders
SET order_status = 'DELIVERED'
WHERE order_status = 'SHIPPED';
```

문제점:

- 락이 오래 유지된다.
- undo 로그가 커진다.
- redo 로그가 커진다.
- 롤백 비용이 커진다.
- 일반 API 응답이 지연될 수 있다.

개선 방식:

```
UPDATE orders
SET order_status = 'DELIVERED'
WHERE order_id BETWEEN ? AND ?
AND order_status = 'SHIPPED';
```

운영 원칙:

- PK 범위 기준으로 나눈다.
- 체크 단위로 커밋한다.
- 실패 지점을 기록한다.
- 재실행 가능하게 만든다.

13-3. 대량 DELETE

로그성 데이터는 row 단위 DELETE보다 파티션 DROP이 유리할 수 있다.

```
ALTER TABLE order_event_log DROP PARTITION p202601;
```

대량 DELETE가 필요하다면 LIMIT 기반으로 나눠서 처리한다.

```
DELETE FROM product_view_log
WHERE created_at < ?
LIMIT 10000;
```

14. 캐시 아키텍처

14-1. 캐시 후보

데이터	캐시 적합도	이유
상품 상세	높음	읽기 많음
카테고리 목록	높음	변경 적음
인기 상품	높음	반복 조회 많음
리뷰 통계	중간	갱신 전략 필요
재고 수량	낮음	정합성 위험
주문 상태	낮음	사용자별 민감 데이터

14-2. 캐시 무효화 전략

캐시는 저장보다 무효화가 어렵다.

상품 수정 시:

상품 DB 업데이트
↓
상품 상세 캐시 삭제
↓
상품 목록 캐시 삭제 또는 TTL 만료 대기

리뷰 작성 시:

리뷰 INSERT
↓
product_stat 갱신 이벤트 발행
↓
상품 상세 캐시 삭제

상품 가격 변경 시:

상품 가격 DB 업데이트
↓
상품 상세 캐시 삭제
↓
상품 목록 캐시 삭제

재고는 캐시하지 않는 것이 기본이다.
재고는 정확성이 중요하므로 DB 조건부 UPDATE를 우선 사용한다.

15. 이벤트와 비동기 처리

15-1. 비동기로 처리할 수 있는 작업

- 주문 완료 이메일 발송
- 주문 완료 SMS 발송
- 결제 완료 알림
- 상품 판매량 통계 갱신
- 리뷰 통계 갱신
- 검색 인덱스 갱신
- 관리자 알림

15-2. Outbox 패턴

DB 트랜잭션과 메시지 발행을 안전하게 연결하기 위해 `outbox_event` 테이블을 둔다.

```
outbox_event
- event_id
- aggregate_type
- aggregate_id
- event_type
- payload
- status
- retry_count
- created_at
- published_at
```

주문 결제 완료 시:

```
DB 트랜잭션 안에서
- payment APPROVED 저장
- order PAID 변경
- outbox_event INSERT

트랜잭션 커밋 후
- Outbox Publisher가 이벤트 발행
- 발행 성공 시 status = PUBLISHED
```

이렇게 하면 DB 저장은 성공했는데 메시지 발행은 실패하는 문제를 재처리할 수 있다.

16. 보안 아키텍처

16-1. 인증

선택지는 크게 두 가지다.

```
Session 기반 인증
JWT 기반 인증
```

초기 쇼핑몰은 세션 기반도 충분히 실용적이다.
모바일 앱, 외부 API, 확장성을 고려하면 JWT를 선택할 수 있다.

16-2. 비밀번호 저장

비밀번호는 절대 평문 저장하지 않는다.

BCryptPasswordEncoder 사용

```
String encodedPassword = passwordEncoder.encode(rawPassword);
```

16-3. 인가

사용자 API와 관리자 API는 명확히 분리한다.

일반 사용자

- 내 정보 조회
- 내 주문 조회
- 내 리뷰 작성

관리자

- 전체 주문 조회
- 상품 등록/수정/삭제
- 배송 상태 변경
- 회원 관리

주문 조회 시 반드시 본인 주문인지 확인한다.

```
public Order getMyOrder(Long memberId, Long orderId) {
    Order order = orderRepository.findById(orderId)
        .orElseThrow();

    if (!order.isOwner(memberId)) {
        throw new AccessDeniedException("본인의 주문만 조회할 수 있습니다.");
    }

    return order;
}
```

16-4. 개인정보 보호

개인정보는 최소한으로 저장한다.

보호 대상:

- 이름
- 전화번호

- 주소
- 이메일
- 결제 관련 식별자

운영 화면에서는 마스킹한다.

홍길동 → 홍*동
010-1234-5678 → 010-****-5678
test@example.com → te**@example.com

17. 배치 아키텍처

17-1. 필요한 배치

- 결제 대기 주문 만료 처리
- 예약 재고 해제
- 결제 상태 동기화
- 배송 상태 대량 반영
- 일별 매출 집계
- 상품별 판매량 집계
- 리뷰 통계 갱신
- 쿠폰 만료 처리
- 포인트 만료 처리
- 오래된 로그 삭제

17-2. 배치 처리 원칙

- 한 번에 너무 많이 처리하지 않는다.
- 청크 단위로 나눈다.
- 각 청크마다 커밋한다.
- 실패 지점을 기록한다.
- 재실행 가능해야 한다.
- 중복 실행되어도 결과가 깨지면 안 된다.

17-3. 배치 실행 이력

batch_job_execution
- execution_id

- job_name
- status
- started_at
- ended_at
- processed_count
- success_count
- fail_count
- error_message

배치 실패 데이터:

- ```
batch_failure
```
- failure\_id
  - execution\_id
  - target\_id
  - reason
  - payload
  - created\_at

---

## 18. 모니터링과 운영 지표

### 18-1. API 지표

- API 요청 수
- API 평균 응답 시간
- API p95 응답 시간
- API p99 응답 시간
- 4xx 에러율
- 5xx 에러율

---

### 18-2. 주문 지표

- 주문 생성 성공 수
- 주문 생성 실패 수
- 재고 부족 실패 수
- 결제 실패 수
- 주문 취소 수
- 결제 완료 후 주문 상태 불일치 건수

### 18-3. 결제 지표

- 결제 승인 성공률
- 결제 승인 실패율
- PG 응답 지연 시간
- PG 콜백 지연 건수
- 중복 콜백 수
- 결제 취소 실패 수

### 18-4. DB 지표

- Slow Query
- Lock Wait Time
- Deadlock Count
- Connection Pool 사용률
- Query Timeout 수
- Transaction Rollback 수

### 18-5. 알림 기준

예시:

- 결제 승인 실패율 5분간 10% 초과
- 주문 생성 API p99 3초 초과
- DB Connection Pool 사용률 90% 초과
- Deadlock 5분간 5회 이상
- PENDING\_PAYMENT 주문이 30분 이상 유지
- payment APPROVED인데 order PAID가 아닌 건수 발생

## 19. 장애 복구 전략

### 19-1. 주문 생성 중 실패

상황:

주문 생성 중 재고 차감 성공 후 order 저장 실패

해결:



같은 DB 트랜잭션 안에서 처리한다.  
order 저장 실패 시 재고 차감도 롤백된다.

## 19-2. 결제 성공 후 서버 장애

상황:

PG 결제 승인 성공  
서버 응답 실패  
사용자는 결제 실패로 인식  
DB에는 주문 상태 PENDING\_PAYMENT

해결:

- payment\_key 기준으로 PG 결제 내역 조회
- 결제 상태 동기화 배치 수행
- payment APPROVED 반영
- order PAID 반영

## 19-3. 콜백 중복 수신

상황:

PG사가 같은 결제 성공 콜백을 여러 번 보냄

해결:

- payment\_key UNIQUE
- 이미 APPROVED 상태면 무시
- 성공 응답 반환

## 19-4. 배치 실패

상황:

배송 상태 100만 건 반영 중 60만 건 처리 후 실패

해결:

- 청크 단위 커밋
- 마지막 성공 PK 기록
- 실패 데이터 기록
- 실패 지점부터 재실행

---

## 20. 테스트 전략

### 20-1. 단위 테스트

```
Product.decreaseStock()
Inventory.reserve()
Inventory.release()
Order.pay()
Order.cancel()
Order.ship()
Review.validateWritable()
Payment.approve()
Payment.cancel()
```

---

### 20-2. 통합 테스트

- 주문 생성 시 orders, order\_item, delivery 저장 확인
- 재고 부족 시 주문 실패 확인
- 결제 성공 시 order PAID 변경 확인
- 결제 실패 시 예약 재고 해제 확인
- 주문 취소 시 결제 취소 및 재고 복구 확인
- 리뷰 중복 작성 실패 확인

---

### 20-3. 동시성 테스트

재고 1개 상품에 동시에 100명이 주문한다.

기대 결과:

성공 주문 수 = 1  
실패 주문 수 = 99  
최종 재고 = 0

여러 상품 주문 데드락 테스트:

요청 1: A, B 상품 주문  
요청 2: B, A 상품 주문

기대:  
- product\_id 오름차순 처리로 데드락 방지

리뷰 중복 작성 테스트:

동시에 같은 order\_item에 리뷰 작성 요청 10개

기대:  
- 성공 1개  
- 실패 9개

## 20-4. 성능 테스트

상품 100만 건 기준 상품 목록 조회 시간  
주문 100만 건 기준 주문 목록 조회 시간  
리뷰 100만 건 기준 리뷰 목록 조회 시간  
동시 주문 요청 1,000건 처리 결과  
대량 UPDATE 실행 중 일반 API 응답 시간

## 21. 운영 체크리스트

### 21-1. 배포 전 체크리스트

- [ ] 주문 상태 전이 규칙이 도메인에 구현되어 있는가?
- [ ] 결제 요청에 idempotency\_key가 있는가?
- [ ] PG 콜백 중복 처리가 가능한가?
- [ ] 결제 성공 후 주문 상태 불일치 복구 배치가 있는가?
- [ ] 재고 차감이 조건부 UPDATE 또는 락으로 보호되는가?
- [ ] 리뷰 중복 작성이 DB UNIQUE 제약조건으로 막히는가?
- [ ] 주문 조회 시 본인 주문 여부를 검증하는가?

- [ ] 관리자 API 권한이 분리되어 있는가?
- [ ] 상품 목록, 주문 목록, 리뷰 목록 인덱스가 설계되어 있는가?
- [ ] 대량 UPDATE/DELETE가 청크 단위로 처리되는가?
- [ ] 캐시 무효화 전략이 정의되어 있는가?
- [ ] 배치 실패 이력이 저장되는가?
- [ ] Slow Query를 추적할 수 있는가?
- [ ] 결제 실패율, 주문 실패율 알림이 있는가?

## 22. 최종 결론

운영 가능한 온라인 쇼핑몰은 단순히 다음 기능을 구현하는 것이 아니다.

회원가입  
 상품 조회  
 주문 생성  
 결제  
 배송  
 리뷰

실제 운영 가능한 구조는 다음을 함께 만족해야 한다.

- 동시에 주문해도 재고가 깨지지 않아야 한다.
- 결제가 중복 처리되지 않아야 한다.
- PG 콜백이 중복으로 와도 안전해야 한다.
- 결제와 주문 상태가 불일치하면 복구할 수 있어야 한다.
- 주문 상태는 명확한 상태 머신으로 관리되어야 한다.
- 대량 데이터 처리는 청크와 배치로 안정적으로 처리해야 한다.
- 조회 API는 인덱스, 페이징, DTO 조회로 최적화해야 한다.
- 캐시는 무효화 전략까지 포함해야 한다.
- 보안, 인가, 개인정보 보호가 적용되어야 한다.
- 운영자는 로그와 모니터링으로 장애 원인을 추적할 수 있어야 한다.

핵심 문장:

쇼핑몰의 핵심은 상품 CRUD가 아니라 주문·결제·재고의 정합성이다.

운영 가능한 아키텍처는 정상 흐름보다 실패 흐름을 더 많이 고려한다.

실무 시스템은 한 번 성공하는 코드가 아니라, 실패해도 복구 가능한 구조로 설계해야 한다.